Round-trip Engineering for Tactical DDD: A Constraint-Based Vision for the Masses

Weixing Zhang
Karlsruhe Institute of Technology
Karlsruhe, Germany
weixing.zhang@kit.edu

Mario Herb
esentri AG
Karlsruhe, Germany
mario.herb@esentri.com

Martin Armbruster
Karlsruhe Institute of Technology
Karlsruhe, Germany
martin.armbruster@kit.edu

Bowen Jiang
Karlsruhe Institute of Technology
Karlsruhe, Germany
bowen.jiang@kit.edu

Marcel Vielsack
esentri AG
Karlsruhe, Germany
marcel.vielsack@esentri.com

Anne Koziolek
Karlsruhe Institute of Technology
Karlsruhe, Germany
koziolek@kit.edu

Abstract

Despite Domain-Driven Design's proven value in managing complex business logic, a fundamental semantic expressiveness gap persists between generic modeling languages and tactical DDD patterns, causing continuous divergence between design intent and implementation. We envision a constraint-based tactical modeling environment that transforms abstract architectural principles into explicit, tool-enforced engineering constraints. At its core is a DDD-native metamodel where tactical patterns are first-class modeling primitives, coupled with a real-time constraint verification engine that prevents architectural violations during modeling, and bidirectional synchronization mechanisms that maintain model-code consistency through round-trip engineering. This approach aims to democratize tactical DDD by embedding expert-level architectural knowledge directly into modeling constraints, enabling small teams and junior developers to build complex business systems without sacrificing long-term maintainability. By lowering the technical barriers to DDD adoption, we envision transforming tactical DDD from an elite practice requiring continuous expert oversight into an accessible engineering discipline with tool-supported verification.

CCS Concepts

• **Do Not Use This Code → Generate the Correct Terms for Your Paper**; *Generate the Correct Terms for Your Paper*; Generate the Correct Terms for Your Paper; Generate the Correct Terms for Your Paper.

Keywords

Tactical Domain-Driven Design, Semantic Modeling, Constraint-Based Engineering, Model-Code Consistency, Round-trip Engineering

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference FSE'26, Montréal, QC, Canada
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/10.1145/3803437.3805566

ACM Reference Format:
Weixing Zhang, Mario Herb, Martin Armbruster, Bowen Jiang, Marcel Vielsack, and Anne Koziolek. 2026. Round-trip Engineering for Tactical DDD: A Constraint-Based Vision for the Masses. In *Proceedings of ACM International Conference on the Foundations of Software Engineering (Conference FSE'26).* ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/3803437.3805566

1 Introduction

Domain-Driven Design (DDD) [5], since its introduction in 2003, has become an established methodology for managing complex business logic. However, the practice of tactical DDD still faces inadequate tool support. A recent systematic literature review [17] that analyzed 36 peer-reviewed studies identified the persistent "model-code gap" as a primary technical challenge in DDD adoption: domain models frequently diverge from their code implementations [24][25], undermining the core promise of DDD to maintain alignment between domain understanding and software structure.

The root cause of this problem lies in the insufficient semantic expressiveness of existing modeling tools. Neither generic UML tools nor code-first frameworks can adequately encode the semantic constraints of DDD patterns, resulting in model validation relying on manual review rather than tool enforcement [17]. This tool-level deficiency is a significant contributor to the model-code gap.

We propose a constraint-based round-trip engineering environment that elevates tactical DDD patterns to first-class citizens of the modeling language, rather than auxiliary annotations to UML. In this environment, *Aggregate Root*, *Entity*, and *Value Object* are no longer class diagram stereotypes that must be manually maintained, but modeling primitives with built-in semantic constraints. For example, when modeling an *aggregate*, if a developer attempts to have an *entity* directly reference an *entity* from another *aggregate*—which violates *aggregate* boundary rules—the verification engine will immediately reject this operation at modeling time and suggest using an identifier *Value Object* instead. By encoding expert-level architectural knowledge as tool-enforced constraints, this vision aims to lower the barrier to tactical DDD, enabling small teams and junior developers to build complex business systems without sacrificing long-term maintainability.

2 Gap Analysis: The Semantic Deficit in DDD Tooling

The practice of tactical DDD faces a structural dilemma in tool support. Existing solutions can be categorized into several approaches:

Generic UML tools exhibit insufficient semantic expressiveness. Tools such as Enterprise Architect and Visual Paradigm represent DDD patterns—*Aggregate Root*, *Entity*, *Value Object*—as stereotypes on class diagrams, which are essentially metadata annotations [17]. This approach cannot encode the semantic constraints of patterns [11, 18]. For example, tools cannot prevent developers from placing *Entity* classes outside their owning *aggregates*, nor can they enforce the immutability of *Value Object* [7, 15]. Core DDD rules, such as *Aggregate* boundary integrity checks and *Entity* lifecycle dependency validation, cannot be automatically verified by tools at modeling time. While tools like JQAssistant [8] can check some DDD rules at compile time, such checks operate at the code level rather than during modeling, and current implementations cover only a subset of possible constraints. Most violations are still discovered during code review or at runtime.

Pure code-first frameworks lack model views. Frameworks such as JMolecules [21] annotate the DDD roles of classes through marker interfaces or annotations, providing partial constraint checking at compile time. However, such solutions have two fundamental deficiencies: first, the absence of visual models prevents domain experts from participating in modeling discussions—domain experts typically do not read code [9], yet core DDD practices such as EventStorming [2] and Context Mapping [5] depend on shared visual artifacts; second, model evolution tracking relies on code commit history, lacking a systematic overview of high-level architectural changes such as Bounded Contexts and Aggregate structures [3].

Strategic alignment alone is insufficient. Practitioners have also attempted to reduce DDD's reliance on expert judgment through strategic-level guidance—for example, using bounded contexts to define microservice boundaries [26]. However, such alignment cannot prevent tactical violations: aggregate boundaries can still be breached and domain invariants compromised within correctly bounded services [26].

In summary, existing tools do not treat DDD patterns as first-class citizens of the modeling language - none is specifically designed for DDD's tactical patterns [12, 19].

3 Vision: DDD-Native Round-trip Engineering

We propose a constraint-based tactical modeling environment realized through four components: 1) the **Domain Modeler** provides a graphical modeling interface based on a native DDD metamodel; 2) the **Domain Model Verifier** performs real-time constraint checking and offers intelligent repair suggestions, recent advances in GenAI also show potential for enhancing automated repair suggestions [14][13]; 3) the **Domain Code Generator** enables model-to-code incremental synchronization; 4) the code-to-model direction leverages the existing **Domain Mirror** component [4]. Together, these transform DDD semantic rules from implicit expert knowledge into tool-enforced constraints. The Domain Modeler provides a graphical modeling interface with real-time visual feedback, aligned with the vision of metamodel-based language workbenches [23].

3.1 Metamodel Hierarchy

In contrast to mapping DDD patterns to generic UML metaclasses, we propose a native DDD metamodel in which the containment relationships and dependency rules among tactical patterns are explicitly encoded as metamodel hierarchies. Specifically: *Aggregate* is the top-level modeling unit that maintains internal consistency through *AggregateRoot* as the sole entry point, explicitly containing *Entity* and *ValueObject*; *Entity* possesses unique identity and lifecycle, while *ValueObject* defines equality through attributes and is immutable; *DomainEvent* is published by *Aggregate* to capture facts about aggregate state changes. This metamodel structure inherently prohibits cross-aggregate entity references, i.e., inter-aggregate associations can only be realized through *ValueObject*-typed identifiers. Additionally, the metamodel includes *Repository* and *DomainService* as architectural connectors: *Repository* abstracts the persistence access of *Aggregates* with a one-to-one correspondence to *Aggregates*; *DomainService* encapsulates domain logic that does not naturally belong to a single *Entity* or *ValueObject* [5]. These elements, together with the core tactical patterns, constitute a complete DDD metamodel that enables constraint verification to cover the entire spectrum from domain modeling to architectural organization.

3.2 Constraint Verification and Intelligent Guidance

Based on the above metamodel, we identify two types of DDD semantic constraints: (1) structural constraints that specify legal composition relationships among patterns (e.g., value objects cannot contain entity references); (2) behavioral constraints that specify the operational semantics and state evolution rules of patterns, to the extent decidable by static analysis (e.g., aggregate root methods are the sole entry point for modifying internal aggregate state). The verification engine checks these constraints in real-time during modeling operations and provides contextualized repair suggestions. For example, when a developer attempts to define a public setter method on an *Entity* that allows external modification of aggregate state bypassing the *AggregateRoot*, the behavioral constraint verifier flags this violation and suggests encapsulating the state change within an *AggregateRoot* method. This tool-level intelligent guidance embeds DDD expert architectural decision knowledge into the modeling workflow, enabling junior developers to naturally acquire tactical DDD best practices. Unlike existing constraint-based approaches that focus on post-hoc validation or pattern-based detection of violations [6], our approach embeds tactical DDD constraints directly into both the modeling language's metamodel and the tool's verification engine, supporting continuous validation during model evolution. While our current repair suggestions are rule-based, recent work on LLM-based co-evolution of DSL definitions and instances [22] suggests potential for more adaptive, context-aware guidance in future iterations.

3.3 Bidirectional Synchronization Mechanism

To bridge the gap between model and code, we adopt a bidirectional synchronization strategy. In the code-to-model direction, we leverage the existing Domain Mirror component [4], which uses reflection (specifically the Java Reflection API) to transfer meta-information from code to the model level. In the model-to-code direction, the Domain Code Generator to be developed will be inspired by the Vitruvius framework [10], a view-based approach for consistency preservation in model-driven development, to maintain view consistency through incremental model transformations. Specifically, model changes will be captured by a delta domain metamodel and translated into corresponding code modifications through declarative transformation rules.

4 Key Research Challenges

Realizing the above vision requires systematically addressing the following core challenges:

4.1 Challenge 1: Balancing Completeness and Practicality of DDD Constraint Rules

Tactical DDD patterns contain well-defined semantic constraints, but there is currently a lack of systematic research on how to translate these constraints into tool-level verification rules. DDD's core rules such as "entities cannot reference across aggregate boundaries" and "value objects must be immutable" can be enforced as hard constraints, however, DDD practice requires consideration of additional factors, including design recommendations proposed by the DDD community (such as Vernon's "design small aggregates" principle [20]) and engineering trade-offs (such as performance optimization and legacy compatibility) that depend on specific contexts. The challenge is designing a tool that ensures semantic rigor without becoming an obstacle to real-world projects. One pragmatic approach is to allow developers to override specific constraints with documented justifications, or to disable certain constraints for specific projects.

4.2 Challenge 2: Incremental Delta Identification and Propagation

The primary challenge of incremental synchronization is identifying model changes. When developers add a new value object attribute to an aggregate, the system must distinguish between an incremental modification and a scenario requiring complete code regeneration. A naive approach would be full regeneration, but this risks of overwriting manually added business logic. DDD patterns make this harder: dependencies exist among patterns, and changes to a single element may violate aggregate boundaries or consistency rules. To address this and in building on the mapping of software architecture models to source code using Vitruvius [16], we plan to define a delta metamodel that explicitly expresses the semantics of model changes, coupled with declarative transformation rules to enable precise mapping from models to code.

4.3 Challenge 3: Round-trip Implementation

Round-trip engineering includes two directions: the model-to-code direction (discussed in Section 4.2) and the code-to-model direction, which is realized through the existing Domain Mirror component [4] that uses Java Reflection API to transfer code structure information to the model level. The key challenge is the consistency guarantee between the two directions, i.e., the structure assumptions when the Domain Code Generator generates code must match the identification logic when the Domain Mirror does reflection. For example, when Domain Code Generator generates an entity class, it assumes the class contains an identifier field, and Domain Mirror must also be able to recognize the correspondence between this field and the entity identifier in the model when doing reflection; if developers manually rename or delete this field, the round-trip synchronization will cause inconsistency between the model and code. This consistency challenge extends beyond DDD-specific tooling; maintaining coherent relationships among software artifacts, running systems, and models is a broader concern in continuous software development [1].

5 Research Roadmap

Based on the components and challenges mentioned above, our Research Roadmap will include the following five phases (Figure 1):

5.1 Phase 1: Metamodel and Initial Modeling

We will formalize tactical DDD elements into a metamodel that accurately represents core concepts such as Aggregates, Entities, and Value Objects. Based on this metamodel, we will develop the Domain Modeler component with its visualization capabilities, along with a repository for persisting domain models.

5.2 Phase 2: Code Generation

We will develop the initial Domain Code Generator component, which enables the generation of a source code representation of a given domain model. Building on the Vitruvius framework [10], we primarily target Java, with other languages to be explored and developed.

5.3 Phase 3: Constraint Verification

Based on existing literature and a literature search, we will identify and formally define the DDD constraint rules, addressing challenge 1 in the process. The resulting constraints will be included in the developed Domain Model Verifier component, which checks the constraints and suggests possibilities for repairing constraint violations.

5.4 Phase 4: Incremental and Round-trip Synchronization

We will enhance the Domain Modeler and Domain Code Generator with incremental and round-trip synchronization, addressing challenges 2 and 3. This involves specifying a delta domain metamodel to express DDD-typical changes, enabling incremental model-to-code transformations that modify only affected code portions. The code-to-model direction leverages the existing Domain Mirror component [4].

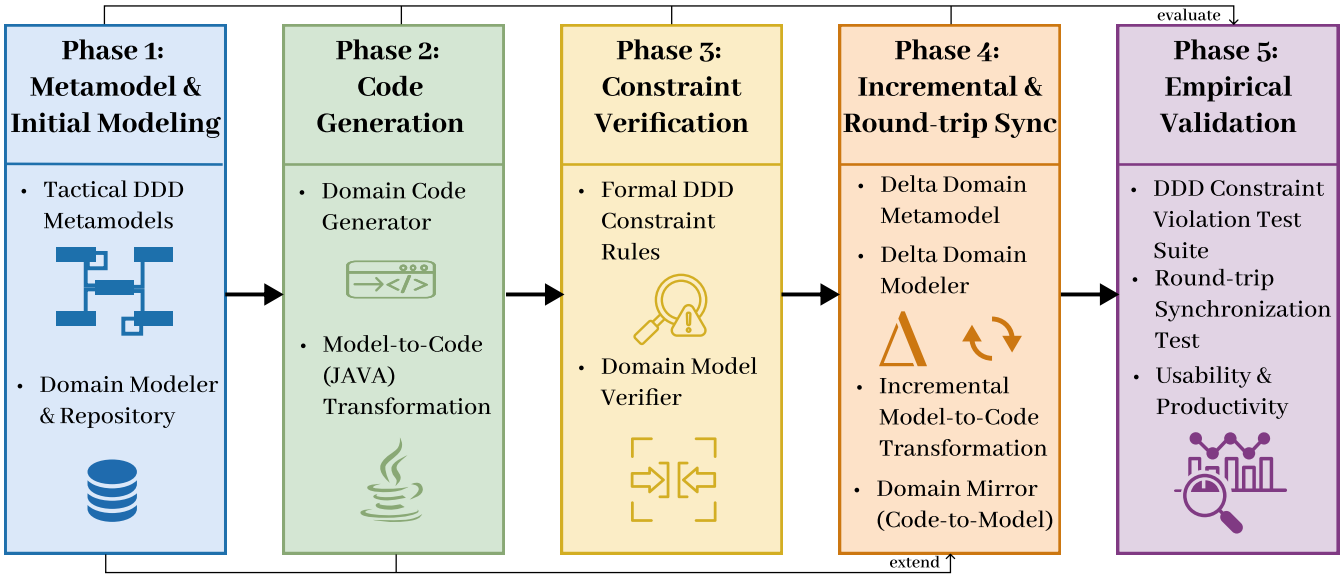

Figure 1: Overview of Research Roadmap

5.5 Phase 5: Empirical Validation

Finally, we plan to empirically evaluate and validate the developed components, as outlined in the next section.

6 Evaluation Strategy

We plan a three-layered evaluation strategy for validation.

Constraint mechanism effectiveness. We construct a test suite containing DDD constraint violations identified in [17], such as cross-aggregate entity references, value object mutability violations, and aggregate boundary leaks. We measure constraint rule coverage, precision/recall rates, and whether repair suggestions are semantically correct (verified through expert review).

Round-trip synchronization quality. Controlled experiments will test whether incremental synchronization preserves semantic equivalence after model changes, prevents overwriting of manually written business logic, and maintains model-code consistency through multiple round-trips.

Usability and productivity. We conduct comparative studies where junior and senior developers complete the same modeling tasks using traditional UML tools versus the constraint-based DDD modeling environment. We evaluate modeling time, task quality (assessed through expert review), and the rate of architectural violations in the resulting models. The hypothesis is that constraint guidance will bring junior developers' modeling quality closer to the senior level. Additionally, preliminary case studies with industry partners will assess whether the tool fits into real development workflows and whether teams find the constraint enforcement helpful or restrictive.

7 Discussion: Impact and Limitations

Our Vision will have an impact on two levels. First, at the engineering practice and methodology level, the constraint-driven modeling environment can lower the entry barrier to tactical DDD—junior developers do not need to master all architectural principles of classic tactical DDD fully, but can avoid deviating from tactical DDD rules through tool-enforced constraints. This enables small teams to build complex business systems without relying on continuous expert review, essentially transforming DDD from an experience-driven practice into a tool-verifiable engineering discipline. Second, at the software engineering research level, this research will provide an empirical case for constraint-driven domain-specific modeling, with methodological relevance to model-driven engineering and domain-specific language research.

This vision has four limitations. First, it focuses on the Java ecosystem and may not apply to other languages. Second, the vision focuses on tactical DDD patterns and does not cover strategic-level bounded context modeling. Third, the tool enforces DDD rules but does not teach DDD concepts. Teams still need a basic understanding of domain modeling. Fourth, practical adoption depends on seamless integration between modeling and coding environments; if developers perceive friction, they may bypass the modeling layer entirely.

References

[1] Martin Armbruster, Manar Mazkatli, and Anne Koziolek. 2025. A process for continuous consistency-aware quality management. (2025).
[2] Alberto Brandolini. 2018. *Introducing EventStorming: An act of Deliberate Collective Learning*. Leanpub. https://leanpub.com/introducing_eventstorming
[3] Susanne Braun, Stefan Deßloch, Eberhard Wolff, Frank Elberzhager, and Andreas Jedlitschka. 2021. Tackling consistency-related design challenges of distributed data-intensive systems: An action research study. In *Proceedings of the 15th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)*. 1–11.

[4] esentri AG. 2024. Domain Mirror. Retrieved September 2025 from https://github.com/esentri/domainlifecycles/tree/main/mirror
[5] Eric Evans. 2004. *Domain-driven design: tackling complexity in the heart of software.* Addison-Wesley Professional.
[6] Sebastian Herwig. [n. d.]. Pattern Matching in Conceptual Modes-A Formal Multi-Modelling Language Approach. ([n. d.]).
[7] Benjamin Hippchen, Pascal Giessler, Roland Steinegger, Michael Schneider, and Sebastian Abeck. 2017. Designing microservice-based applications by using a domain-driven design approach. *International Journal on Advances in Software* 10, 3&4 (2017), 432–445.
[8] jqassistant. 2025. jQAssistant. https://github.com/jqassistant. Accessed: 2026-01-22.
[9] Stefan Kapferer and Olaf Zimmermann. 2020. Domain-driven service design: Context modeling, model refactoring and contract generation. In *Symposium and Summer School on Service-Oriented Computing*. Springer, 189–208.
[10] Heiko Klare, Max E Kramer, Michael Langhammer, Dominik Werle, Erik Burger, and Ralf Reussner. 2021. Enabling consistency in view-based system development—the vitruvius approach. *Journal of Systems and Software* 171 (2021), 110815.
[11] Duc Minh Le, Duc-Hanh Dang, and Viet-Ha Nguyen. 2018. On domain driven design using annotation-based domain specific language. *Computer Languages, Systems & Structures* 54 (2018), 199–235.
[12] Duc Minh Le, Duc-Hanh Dang, and Ha Thanh Vu. 2019. jDomainApp: a module-based domain-driven software framework. In *Proceedings of the 10th International Symposium on Information and Communication Technology*. 399–406.
[13] Jialong Li, Mingyue Zhang, Nianyu Li, Danny Weyns, Zhi Jin, and Kenji Tei. 2024. Exploring the Potential of Large Language Models in Self-adaptive Systems. In *Proceedings of the 19th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. 77–83. doi:10.1145/3643915.3644088
[14] Jialong Li, Mingyue Zhang, Nianyu Li, Danny Weyns, Zhi Jin, and Kenji Tei. 2024. Generative AI for Self-Adaptive Systems: State of the Art and Research Roadmap. *ACM Transactions on Autonomous and Adaptive Systems* 19, 3, Article 13 (Sept. 2024), 60 pages. doi:10.1145/3686803
[15] Gururaj Maddodi, Slinger Jansen, and Michiel Overeem. 2020. Aggregate architecture simulation in event-sourcing applications using layered queuing networks. In *Proceedings of the ACM/SPEC International Conference on Performance Engineering*. 238–245.
[16] Manar Mazkatli, David Monschein, Martin Armbruster, Robert Heinrich, and Anne Koziolek. 2025. Continuous integration of architectural performance models with parametric dependencies–the CIPM approach. *Automated Software Engineering* 32, 2 (2025), 54.
[17] Ozan Özkan, Önder Babur, and Mark van den Brand. 2025. Domain-Driven Design in software development: A systematic literature review on implementation, challenges, and effectiveness. *Journal of Systems and Software* (2025), 112537.
[18] José Roberto C Perillo, Eduardo M Guerra, and Clovis T Fernandes. 2009. Daileon: a tool for enabling domain annotations. In *Proceedings of the Workshop on AOP and Meta-Data for Software Evolution*. 1–4.
[19] Monique Snoeck and Yves Wautelet. 2022. Agile MERODE: a model-driven software engineering method for user-centric and value-based development. *Software and Systems Modeling* 21, 4 (2022), 1469–1494.
[20] Vaughn Vernon. 2013. *Implementing domain-driven design.* Addison-Wesley.
[21] xMolecules. 2020. JMolecules Project. Retrieved November 2025 from https://github.com/xmolecules/jmolecules Accessed November, 2025.
[22] Weixing Zhang, Bowen Jiang, Yuhong Fu, Anne Koziolek, Regina Hebig, and Daniel Strüber. 2026. Leveraging LLMs to support co-evolution between definitions and instances of textual DSLs: A Systematic Evaluation. *arXiv preprint arXiv:2602.11904* (2026).
[23] Weixing Zhang, Jan-Philipp Steghöfer, Regina Hebig, and Daniel Strüber. 2023. A rapid prototyping language workbench for textual DSLs based on Xtext: Vision and progress. *arXiv preprint arXiv:2309.04347* (2023).
[24] Wenli Zhang, Weixing Zhang, Daniel Strüber, and Regina Hebig. 2023. Manual abstraction in the wild: A multiple-case study on oss systems' class diagrams and implementations. In *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, 36–46.
[25] Wenli Zhang, Weixing Zhang, Daniel Strüber, and Regina Hebig. 2025. An empirical study of manual abstraction between class diagrams and code of open-source systems. *Software and Systems Modeling* 24, 6 (2025), 1797–1823.
[26] Chenxing Zhong, Shanshan Li, Huang Huang, Xiaodong Liu, Zhikun Chen, Yi Zhang, and He Zhang. 2024. Domain-driven design for microservices: An evidence-based investigation. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1425–1449.

Received 22 January 2026; accepted 17 March 2026